

BookNest

Votre collection personnelle, comme une vraie bibliothèque

Bibliothèque personnelle unifiée — suivi, emprunts et lecture en un seul outil



PHP • MVC • PostgreSQL • Redis • Swagger

Résumé

BookNest est une application web de bibliothèque personnelle qui réunit dans un seul outil et une seule base de données trois usages habituellement dispersés : le catalogage des livres possédés, le suivi des prêts consentis à des proches, et la découverte de livres du domaine public lus directement dans l'application via l'API Gutendex (Project Gutenberg).

Le projet repose sur une architecture MVC maison en PHP 8.2 (sans framework), une base PostgreSQL 13, un cache Redis dont le client a été réimplémenté directement au-dessus d'un socket TCP (protocole RESP), une authentification JWT, et une conteneurisation Docker Compose complète (PHP-FPM, nginx, PostgreSQL, Redis). L'API est documentée via Swagger/OpenAPI généré depuis des annotations PHPDoc.

Ce dossier présente le cahier des charges, la conception Merise complète (MCD, MLD, MPD), l'architecture logicielle, la sécurité, les tests, la stratégie Git et le déploiement, avec des extraits de code réels commentés ligne à ligne.

Abstract

BookNest is a personal library web application unifying three usually separate concerns in a single tool and database: cataloguing owned books, tracking physical loans to friends and family, and discovering public-domain books read directly in-app via the Gutendex (Project Gutenberg) API. It is built on a hand-rolled PHP 8.2 MVC architecture, PostgreSQL 13, a Redis cache client re-implemented over a raw TCP socket (RESP protocol), JWT authentication, and a full Docker Compose containerisation (PHP-FPM, nginx, PostgreSQL, Redis), with the API documented via Swagger/OpenAPI.

Sommaire

Résumé.....	2
Abstract.....	2
Sommaire	3
Liste des figures et des tableaux	7
Figures.....	7
Glossaire.....	7
Partie I — Présentation générale	8
Chapitre 1 — Introduction générale.....	8
1.1 Présentation personnelle.....	8
1.2 Contexte du projet	8
1.3 Problématique	8
1.4 Objectifs.....	9
1.5 Méthodologie de travail	9
1.6 Gestion de projet — tableau Trello	9
Chapitre 2 — Étude préalable	10
2.1 Étude de l'existant	10
2.2 Critique de l'existant.....	10
2.3 Solution proposée : BookNest	10
2.4 Public cible et cas d'usage	10
Partie II — Cahier des charges	11
Chapitre 3 — Cahier des charges fonctionnel	11
3.1 Acteurs du système	11
3.2 Besoins fonctionnels	11
3.3 Besoins non-fonctionnels	11
3.4 Diagramme des cas d'utilisation	11
Chapitre 4 — Cahier des charges technique.....	12
4.1 Contraintes techniques.....	12
4.2 Choix technologiques justifiés	12
4.3 Environnements	12
Partie III — Conception Merise	13
Chapitre 5 — Dictionnaire de données	13
5.1 Recensement des entités.....	13
Chapitre 6 — MCD (Modèle Conceptuel de Données)	14
6.1 Règles de gestion	14
6.2 Diagramme MCD.....	14

6.3 Explication des entités et associations	14
Chapitre 7 — MLD (Modèle Logique de Données)	15
7.1 Règles de passage MCD → MLD	15
7.2 Schéma MLD	15
author, started_at, ended_at)	15
7.3 Description des relations	15
Chapitre 8 — MPD (Modèle Physique de Données)	16
8.1 Types PostgreSQL retenus	16
8.2 Schéma MPD	16
8.3 Index et contraintes	16
Chapitre 9 — Architecture MVC	17
9.2 Diagramme d'architecture	17
9.3 Flux d'une requête HTTP	17
9.3 Le routeur	18
18	
9.4 Modèles	18
18	
Chapitre 10 — Structure des dossiers	20
10.1 Arborescence du projet	20
10.2 Rôle de chaque dossier	20
Chapitre 11 — Base de données PostgreSQL	21
11.1 Connexion et migrations	21
11.2 Transactions	21
Chapitre 12 — Cache Redis	22
12.1 Stratégie de cache	22
12.2 Un client RESP réimplémenté	22
Chapitre 13 — Sécurité et Authentification	23
13.1 JWT + session hybride	23
23	
13.2 Middleware de protection des routes	23
23	
13.3 Sécurité des données	23
Chapitre 14 — API et Documentation Swagger	24
14.1 Génération depuis les annotations	24
14.2 Endpoints principaux	25
Partie V — Front-end BookNest	26
Chapitre 15 — Direction artistique	26
15.1 Une identité assumée	26

15.2 Zoning	26
15.3 Wireframes	26
15.4 Palette de couleurs	27
15.5 Prototype	28
Chapitre 16 — Écrans principaux	31
16.1 Page d'accueil — étagères	31
16.2 Fiche livre	31
16.3 Lecteur immersif	32
16.4 Formulaire d'ajout	32
16.5 Page de connexion/Inscription	33
Partie VI — Qualité & Livraison	34
Chapitre 17 — Tests	34
17.1 Tests automatisés	34
17.2 Cahier de recette	34
Chapitre 18 — Gestion Git	35
18.1 Organisation des branches	35
18.2 Convention de commits	35
Chapitre 19 — Déploiement	36
19.1 Docker Compose	36
19.2 Migrations versionnées	37
19.3 Hébergement en ligne et intégration continue	37
Partie VII — Bilan	38
Chapitre 20 — Difficultés rencontrées et solutions	38
20.1 Incompatibilité Docker Desktop / macOS	38
20.2 Bascule d'API externe pour le catalogue libre service	38
20.3 Cache navigateur masquant les mises à jour CSS	38
20.4 Perte de la base de données après renommage du projet	38
20.5 Doublons de livres importés depuis le catalogue	39
20.6 Lenteur perçue de la recherche et de la lecture	39
20.7 Cache Redis silencieusement inactif	39
20.8 Corruption du cache sur les grosses valeurs (texte de livre)	39
Chapitre 21 — Perspectives d'évolution	40
Conclusion générale	40
Bibliographie	41
Conclusion générale	42
Bibliographie	Error! Bookmark not defined.
Annexes	Error! Bookmark not defined.

Liste des figures et des tableaux

Figures

- Figure 1 — MCD de BookNest
- Figure 2 — MLD de BookNest
- Figure 3 — MPD PostgreSQL et ERD
- Figure 4 — Architecture MVC
- Figure 5 — Zoning : connexion/inscription et bibliothèque
- Figure 6 — Zoning : catalogue libre service, fiche livre, lecteur
- Figure 7 — Wireframes : connexion, bibliothèque, catalogue, fiche livre
- Figure 8 — Wireframe du lecteur
- Figure 9 — Page de connexion
- Figure 10 — Page d'accueil (étagères)
- Figure 11 — Fiche livre (modal)
- Figure 12 — Catalogue libre service
- Figure 13 — Lecteur intégré
- Figure 14 — Résultats des tests unitaires
- Figure 15 — Interface Swagger UI

Glossaire

- MVC — Model, View, Controller. Séparation des responsabilités.
- PDO — PHP Data Objects. Couche d'accès aux bases de données.
- JWT — JSON Web Token. Authentification stateless signée.
- RESP — REdis Serialization Protocol. Protocole de communication avec Redis.
- TTL — Time To Live. Durée de vie d'une clé de cache.
- UUID — Universally Unique Identifier. Identifiant unique généré aléatoirement.
- ENUM — Type énuméré PostgreSQL, restreint à une liste de valeurs fixes.
- ORM — Object-Relational Mapping (non utilisé dans BookNest, accès direct via PDO).

Partie I — Présentation générale

Chapitre 1 — Introduction générale

1.1 Présentation personnelle

Je m'appelle Chaima Bekhaouda. J'ai obtenu un baccalauréat STI2D, spécialité SIN (Systèmes d'Information et Numérique), au lycée du Rempart. J'ai ensuite poursuivi vers une licence LLCER (Langues, Littératures et Civilisations Étrangères et Régionales), en anglais, arabe et chinois — un parcours volontairement éloigné de l'informatique à l'époque, mais qui m'a donné une aisance certaine à naviguer entre plusieurs langues et cultures, une compétence qui reste utile dans un secteur aussi internationalisé que le développement logiciel.

Après cette licence, j'ai travaillé pendant un an. C'est à ce moment que j'ai décidé de me réorienter vers l'informatique — un domaine qui m'a toujours attirée, et vers lequel je voulais depuis longtemps consacrer davantage de temps et d'énergie plutôt que de le laisser en simple curiosité personnelle.

Je suis aujourd'hui étudiante à La Plateforme, dans le cadre d'un Bachelor de trois ans, et alternante depuis deux ans chez Technip Energies, où j'apprends et je travaille en parallèle. Technip Energies est une entreprise française de technologies et d'ingénierie de premier plan dans les infrastructures énergétiques et la décarbonation (GNL, hydrogène, chimie durable, gestion du CO₂), employant plus de 18 000 collaborateurs dans 35 pays. Cette alternance m'a permis de confronter très tôt mes apprentissages théoriques à un contexte professionnel réel, au sein d'un grand groupe.

Je souhaite poursuivre mon parcours par un Master (MSc) à Epitech, où je suis déjà inscrite, toujours en alternance — mon entreprise actuelle a confirmé me garder pour les deux prochaines années dans ce cadre. À plus long terme, mon objectif est de poursuivre vers un doctorat à l'étranger, en lien avec Epitech.

1.2 Contexte du projet

Le marché du suivi de lecture est aujourd'hui dominé par des plateformes comme Goodreads (plus de 150 millions de membres, propriété d'Amazon) ou des alternatives plus récentes comme StoryGraph. Ces outils permettent de cataloguer ce qu'on a lu, mais aucun ne couvre l'ensemble réel de la vie d'un lecteur : le suivi des prêts physiques est absent des plus grandes plateformes, et surtout, aucune application de suivi ne permet de lire un livre directement en son sein — la lecture reste systématiquement déléguée à un autre outil.

1.3 Problématique

Comment réunir dans un seul outil trois usages qui vivent aujourd'hui dans des applications séparées — le catalogue personnel, le suivi des prêts, et la lecture de livres gratuits — sans sacrifier la rigueur technique attendue d'une application professionnelle (architecture en couches, base relationnelle, cache, sécurité, documentation d'API) ?

1.4 Objectifs

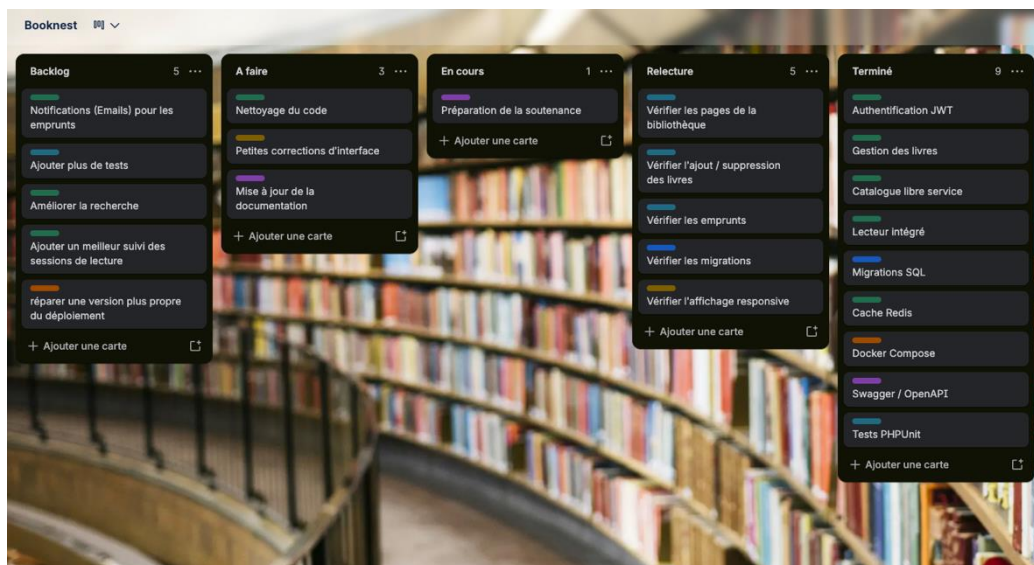
- Offrir une étagère visuelle plutôt qu'un simple tableau de données.
- Gérer la collection : ajout, édition, statuts (à lire, en cours, terminé, prêté).
- Suivre les emprunts : qui a le livre, depuis quand, rendu ou non.
- Donner accès à un catalogue de livres du domaine public, lisibles directement dans l'application.
- Fournir une API documentée via Swagger.

1.5 Méthodologie de travail

Le projet a été développé en solo, en itérations continues : chaque fonctionnalité (emprunts, catalogue libre-service, lecteur, tableau de bord) a été construite, testée manuellement, puis affinée sur retour d'usage immédiat avant de passer à la suivante. Le versioning suit une convention de commit atomiques (feat/fix/chore) sur des branches de fonctionnalité dédiées. La conception de la base de données s'appuie sur Merise, formalisée avant toute écriture de SQL.

1.6 Gestion de projet — tableau Trello

Le suivi des tâches s'est fait via un Trello à 5 colonnes (Backlog, À faire, En cours, Relecture, Terminé), avec des étiquettes par nature de tâche (Backend, Frontend, BDD, Design, Doc, Tests, Infra) :



Ce découpage en 5 étapes (plutôt que la simple paire À faire/Fait) permet d'isoler une étape de relecture avant de considérer une tâche réellement terminée — une discipline utile même en développement solo, pour revenir à froid sur un morceau de code avant de le clore définitivement.

Chapitre 2 — Étude préalable

2.1 Étude de l'existant

Plusieurs solutions ont été étudiées : Goodreads (social, très large catalogue, mais aucune gestion des prêts ni lecture intégrée), StoryGraph (recommandations par ambiance/rythme plutôt que par étoiles, même limite sur la lecture), BookBuddy et Bookly (gestion fine du catalogue et des prêts, mais lecture explicitement hors du périmètre de l'application — Bookly précise d'ailleurs dans sa propre description que le contenu du livre n'est pas géré dans l'app), et Fable (lecteur intégré, mais réservé au contenu sous licence payante via son partenariat Everand).

2.2 Critique de l'existant

- Aucune plateforme ne combine catalogue possédé, prêts et lecture réelle dans un seul outil.
- Le suivi des prêts, quand il existe, reste secondaire et absent des plus grandes plateformes.
- Aucune passerelle entre ce qu'on possède, ce qu'on a prêté et ce qu'on lit gratuitement : chaque source vit dans un outil différent.

2.3 Solution proposée : BookNest

BookNest ne cherche pas à concurrencer Goodreads sur l'échelle ou StoryGraph sur les statistiques sociales. Sa proposition de valeur est l'unification : qu'un livre soit ajouté manuellement, marqué comme prêté, ou importé depuis le catalogue libre-service, il atterrit dans la même table, alimentant les mêmes statistiques et le même fil d'activité — une trace complète de la vie de lecteur de l'utilisateur.

2.4 Public cible et cas d'usage

Lecteurs particuliers gérant leur bibliothèque personnelle, sans dimension communautaire (pas d'amis, pas d'avis publics — l'application est pensée comme un outil privé). Cas d'usage principaux : cataloguer sa collection, suivre qui a emprunté quel livre, découvrir et lire un classique du domaine public sans installer de liseuse dédiée.

Partie II — Cahier des charges

Chapitre 3 — Cahier des charges fonctionnel

3.1 Acteurs du système

BookNest est pensé pour un utilisateur unique par compte : le lecteur, propriétaire de sa collection. Il n'existe pas de rôle visiteur public ni d'administrateur distinct dans le périmètre actuel du projet.

3.2 Besoins fonctionnels

BookNest doit permettre : l'inscription et l'authentification par email/mot de passe ; la création, l'édition de statut et la suppression de livres ; l'organisation en étagères paginées ; la recherche et le filtrage par genre et statut ; l'enregistrement d'emprunts avec nom de l'emprunteur et date de retour ; la recherche et l'ajout de livres du domaine public depuis Gutendex ; la lecture paginée d'un livre directement dans l'application ; la consultation de statistiques et d'un fil d'activité récente.

3.3 Besoins non-fonctionnels

- Performance : les statistiques et listes coûteuses sont mises en cache (Redis, TTL 5 minutes).
- Sécurité : JWT, hachage des mots de passe (password_hash), requêtes préparées PDO systématiques.
- Ergonomie : interface responsive, identité visuelle propre plutôt qu'un CRUD générique.
- Maintenabilité : architecture en couches strictes (Controller / Repository / Service), tests unitaires.
- Portabilité : environnement entièrement conteneurisé (Docker Compose), reproductible sur toute machine.

3.4 Diagramme des cas d'utilisation

Trois grandes familles de cas d'utilisation : la gestion de la collection (ajout, statut, suppression de livres), la gestion des emprunts (prêter, marquer comme rendu), et la découverte/lecture (rechercher, ajouter et lire un livre du domaine public). L'utilisateur authentifié est l'acteur central de ces trois familles.

Chapitre 4 — Cahier des charges technique

4.1 Contraintes techniques

- PHP 8.2-FPM (image Docker officielle) — extensions pdo_pgsql, mbstring, zip, gd, opcache, redis (PECL).
- PostgreSQL 13 comme base de données primaire.
- Redis 7 comme cache, jamais comme stockage métier primaire.
- nginx (stable-alpine) comme serveur web frontal.
- Composer pour la gestion des dépendances (firebase/php-jwt, zircote/swagger-php, phpunit/phpunit).

4.2 Choix technologiques justifiés

Le choix d'une architecture MVC maison plutôt qu'un framework (Laravel, Symfony) a été délibéré : comprendre et maîtriser chaque mécanisme (routage, cycle requête/réponse, middleware) plutôt que de dépendre d'une couche d'abstraction. PostgreSQL a été retenu pour sa robustesse et son typage strict (UUID natifs, types ENUM personnalisés). Redis accélère les statistiques et listes fréquemment lues, avec un client RESP réimplémenté directement au-dessus d'un socket TCP brut — choix pédagogique pour comprendre le protocole sous-jacent plutôt que de dépendre de l'extension PECL comme boîte noire. Swagger documente l'API directement depuis des annotations, gardant code et documentation synchronisés.

4.3 Environnements

Le développement s'est fait sur macOS. Docker Desktop s'étant révélé incompatible avec la version de macOS de la machine de développement (Ventura 13.7.8, hors de la fenêtre de compatibilité de la dernière version de Docker Desktop), l'environnement repose sur Colima, une alternative gratuite fournissant le moteur Docker via une VM Lima légère, sans contrainte de version d'OS.

Partie III — Conception Merise

Chapitre 5 — Dictionnaire de données

5.1 Recensement des entités

Quatre entités structurent le domaine : UTILISATEUR, LIVRE, EMPRUNT et SESSION_LECTURE.

5.2 Tableau du dictionnaire

Attribut	Entité	Type	Contraintes
id	users	UUID	PK, gen_random_uuid()
email	users	VARCHAR(180)	UNIQUE, NOT NULL
password_hash	users	TEXT	NOT NULL, défaut "
extra_shelves	users	INT	NOT NULL, défaut 0
title / author	books	VARCHAR	NOT NULL
status	books	reading_status (ENUM)	to_read / reading / finished / lent
cover_path	books	VARCHAR(255)	chemin de la couverture uploadée
google_volume_id	books	VARCHAR(64)	référence Gutendex si applicable
borrower	loans	VARCHAR(180)	NOT NULL (texte libre)
returned_at	loans	TIMESTAMPTZ	NULL tant que non rendu
started_at / ended_at	reading_sessions	TIMESTAMPTZ	NOT NULL / NULLABLE

Chapitre 6 — MCD (Modèle Conceptuel de Données)

6.1 Règles de gestion

- Un utilisateur possède zéro à plusieurs livres ; un livre appartient à exactement un utilisateur.
- Un livre peut faire l'objet de zéro à plusieurs emprunts.
- Un livre peut faire l'objet de zéro à plusieurs sessions de lecture.
- L'emprunteur n'est pas un utilisateur du système : c'est un attribut texte libre de l'association EMPRUNT.

6.2 Diagramme MCD

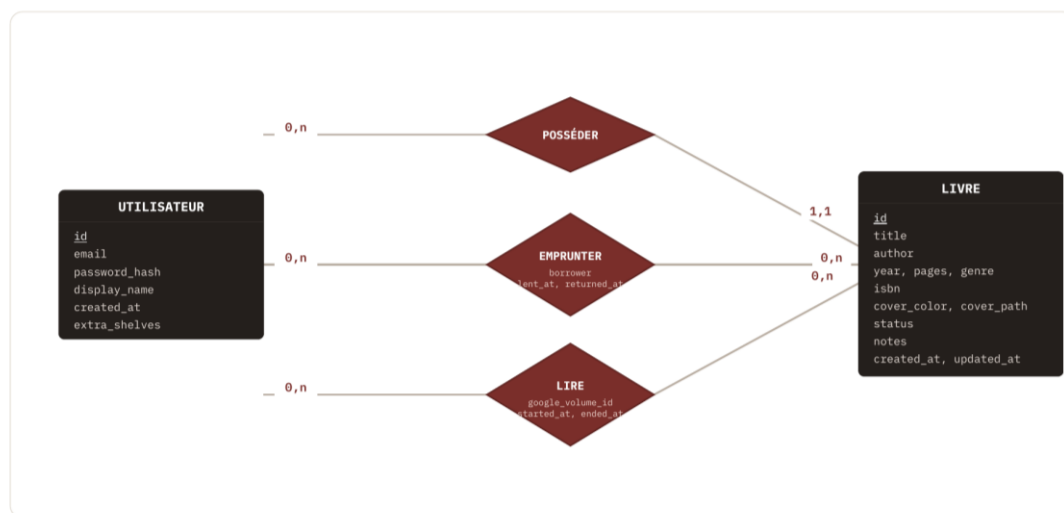


Figure 1 — MCD de BookNest

6.3 Explication des entités et associations

L'entité UTILISATEUR porte l'identification et le compteur d'étagères supplémentaires (extra_shelves). L'association POSSÈDE (1,1 / 0,N) relie UTILISATEUR et LIVRE. ENREGISTRE relie UTILISATEUR et EMPRUNT ; CONCERNE relie LIVRE et EMPRUNT. Cette dernière association est en réalité dénormalisée au niveau physique (la clé utilisateur est reportée directement dans EMPRUNT plutôt que déduite via LIVRE) pour accélérer la requête « tous mes emprunts en cours » sans jointure.

Chapitre 7 — MLD (Modèle Logique de Données)

7.1 Règles de passage MCD → MLD

Chaque entité devient une table. Chaque association (1,1)–(0,N) reporte la clé primaire du côté 1 vers le côté N sous forme de clé étrangère.

7.2 Schéma MLD

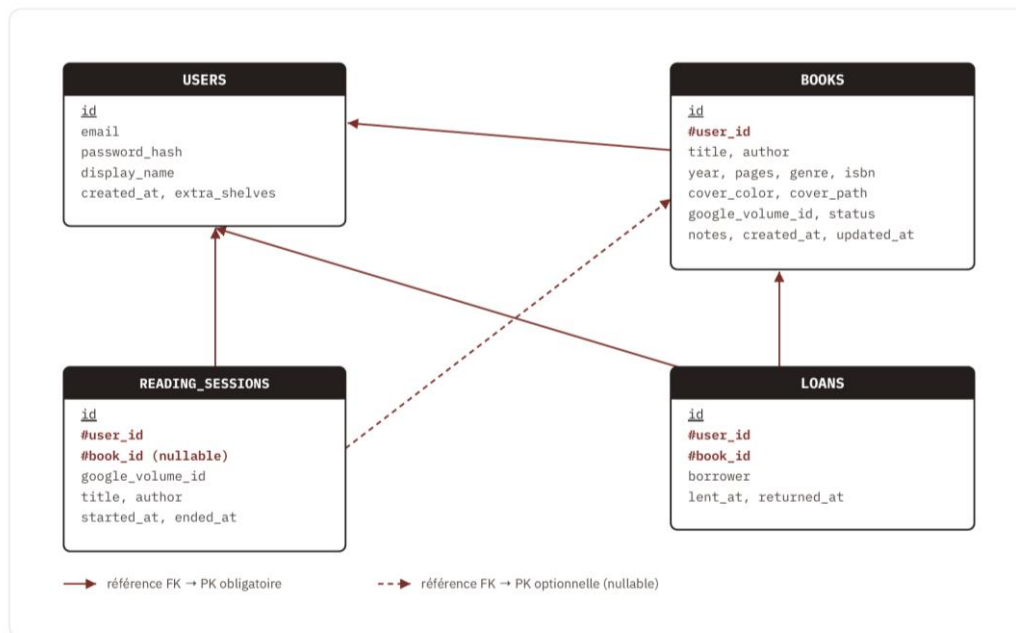


Figure 2 — MLD de BookNest

UTILISATEUR(id, email, password_hash, display_name, created_at, extra_shelves)

LIVRE(id, #user_id, title, author, year, pages, genre, isbn,
cover_color, cover_path, google_volume_id, status, notes,
created_at, updated_at)

EMPRUNT(id, #user_id, #book_id, borrower, lent_at, returned_at)

SESSION_LECTURE(id, #user_id, #book_id, google_volume_id, title,
author, started_at, ended_at)

7.3 Description des relations

Toutes les clés étrangères appliquent ON DELETE CASCADE : la suppression d'un utilisateur supprime automatiquement ses livres, emprunts et sessions de lecture, garantissant l'intégrité référentielle sans logique applicative additionnelle.

Chapitre 8 — MPD (Modèle Physique de Données)

8.1 Types PostgreSQL retenus

- UUID pour toutes les clés primaires, générées via `gen_random_uuid()` (extension `pgcrypto`).
- ENUM `reading_status` pour le statut du livre (`to_read`, `reading`, `finished`, `lent`).
- `TIMESTAMPTZ` pour tout horodatage, avec fuseau.

8.2 Schéma MPD

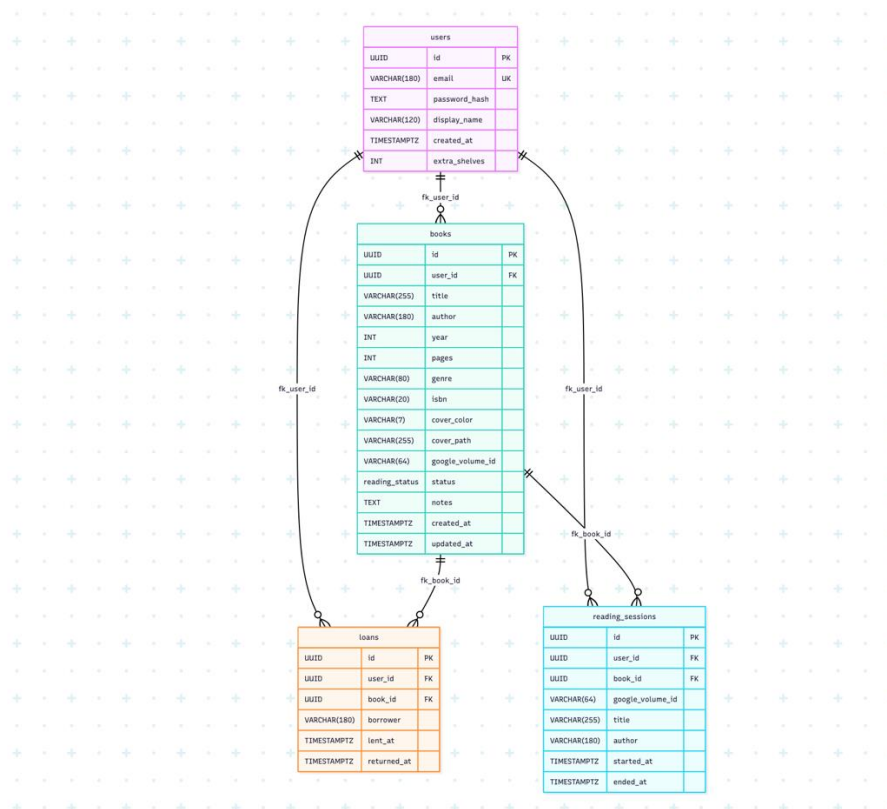


Figure 3 — MPD PostgreSQL

8.3 Index et contraintes

Le schéma évolue via des migrations SQL numérotées séquentiellement (001 à 010), rejouables intégralement sur une base vierge. Trois index accélèrent les requêtes fréquentes : `idx_books_user` (livres d'un utilisateur), `idx_books_status` (filtrage par statut), `idx_loans_user_active` (emprunts en cours d'un utilisateur). Principe retenu : ne jamais modifier une migration déjà appliquée — toute correction de schéma passe par une nouvelle migration.

Chapitre 9 — Architecture MVC

Le pattern Model-View-Controller sépare les responsabilités : le Repository encapsule l'accès aux données (aucune requête SQL n'est jamais construite directement dans un Controller) ; le Controller reçoit la requête, orchestre Repository et Service, prépare la réponse ; la View, un simple gabarit PHP, se contente d'afficher les données transmises.

MVC est un patron d'architecture, pas un outil : n'importe quel framework PHP (Laravel, Symfony...) l'implémente en fournissant déjà le routeur, la couche Request/Response et les middlewares. BookNest fait le choix inverse — aucun framework n'est utilisé. Router.php, Request.php, Response.php et AuthenticateMiddleware.php sont écrits entièrement à la main, sans dépendre d'une couche d'abstraction existante. Le patron est le même que dans un framework ; c'est la mécanique derrière qui est ici assumée directement, dans un but pédagogique : comprendre précisément comment une requête est routée et comment un middleware l'intercepte, plutôt que de s'appuyer sur des conventions déjà résolues par un tiers.

9.2 Diagramme d'architecture

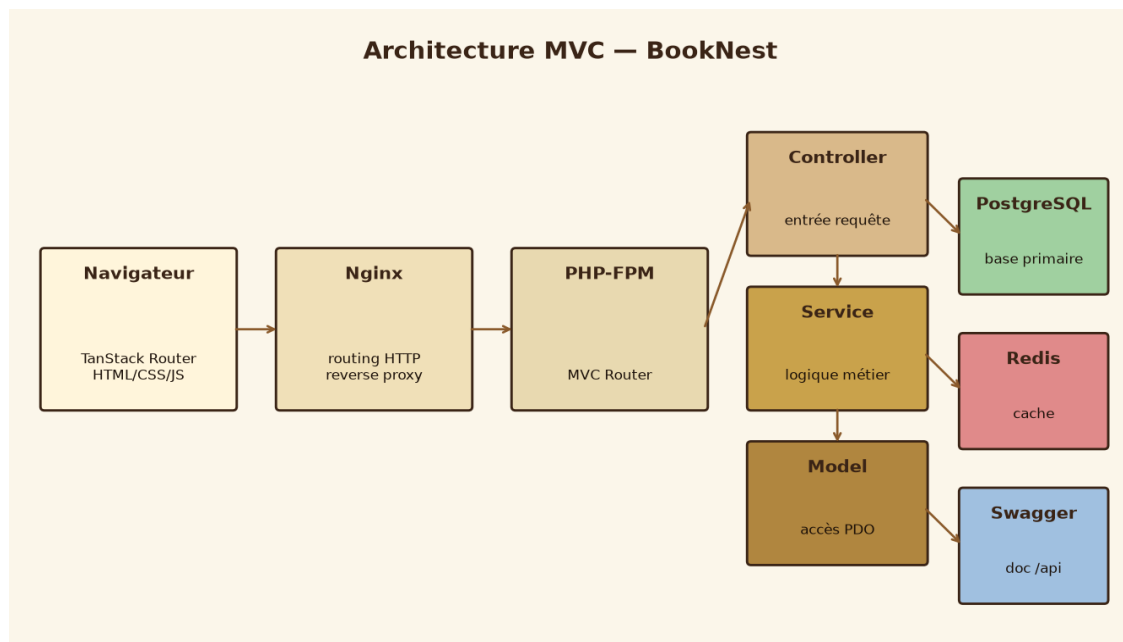


Figure 4 — Architecture MVC de BookNest

9.3 Flux d'une requête HTTP

nginx reçoit la requête et la relaie à PHP-FPM. Le Router de BookNest identifie la route par correspondance exacte de chemin, exécute le middleware d'authentification si la route est protégée, instancie le Controller, qui délègue au Repository correspondant. Le résultat est transmis à la View pour rendu HTML — pas d'API JSON séparée pour les pages, seules quelques routes exposent du JSON pur (`/db-test`).

9.3 Le routeur

```
public function dispatch(string $uri, string $method): void
{
    $uri = parse_url($uri, PHP_URL_PATH) ?? '/';

    foreach ($this->routes[$method] ?? [] as $path => $callback) {
        if ($path === $uri) {
            $callback();
            return;
        }
    }

    http_response_code(404);
    echo '404 - Page not found';
}
```

Explication : Le matching est strict (comparaison exacte de chaîne), sans support de segments dynamiques type `/books/{id}`. Cette limite assumée a orienté certaines routes vers des paramètres en query string plutôt qu'en chemin (par exemple `/library/read?volume_id=...` plutôt que `/library/read/{id}`).

9.4 Modèles

Contrairement aux Repositories (logique d'accès aux données propre à un contrôleur), les Models exposent des méthodes statiques réutilisables autour d'une entité, notamment pour l'authentification :

```
public static function findByEmail(string $email): ?array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('SELECT * FROM users WHERE email = :email LIMIT 1');
    $stmt->execute(['email' => $email]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    return $user ? : null;
}

public static function findById(string $id): ?array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('SELECT id, email, display_name, created_at FROM users WHERE id = :id LIMIT 1');
    $stmt->execute(['id' => $id]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    return $user ? : null;
}
```

Explication : `User::findByEmail()` est utilisé par `AuthController::login()` pour vérifier les identifiants. `User::create()` utilise la clause `RETURNING` de PostgreSQL pour récupérer l'enregistrement fraîchement inséré en un seul aller-retour base, sans `SELECT` supplémentaire.

```
public static function allByUser(string $userId): array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('
        SELECT id, user_id, title, author, year, pages, genre, isbn, cover_color, status, notes, created_at, updated_at
        FROM books
        WHERE user_id = :user_id
        ORDER BY created_at DESC
    ');
    $stmt->execute(['user_id' => $userId]);

    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

public static function findById(string $id, string $userId): ?array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('
        SELECT id, user_id, title, author, year, pages, genre, isbn, cover_color, status, notes, created_at, updated_at
        FROM books
        WHERE id = :id AND user_id = :user_id
        LIMIT 1
    ');
    $stmt->execute([
        'id' => $id,
        'user_id' => $userId
    ]);

    $book = $stmt->fetch(PDO::FETCH_ASSOC);
    return $book ?? null;
}
```

```
public static function create(array $data): array
{
    $pdo = Database::connect();
    $stmt = $pdo->prepare('
        INSERT INTO books (user_id, title, author, year, pages, genre, isbn, cover_color, status, notes)
        VALUES (:user_id, :title, :author, :year, :pages, :genre, :isbn, :cover_color, :status, :notes)
        RETURNING id, user_id, title, author, year, pages, genre, isbn, cover_color, status, notes, created_at, updated_at
    ');

    $stmt->execute([
        'user_id' => $data['user_id'],
        'title' => $data['title'],
        'author' => $data['author'],
        'year' => $data['year'] ?? null,
        'pages' => $data['pages'] ?? null,
        'genre' => $data['genre'] ?? null,
        'isbn' => $data['isbn'] ?? null,
        'cover_color' => $data['cover_color'] ?? '#7a2e2a',
        'status' => $data['status'] ?? 'to_read',
        'notes' => $data['notes'] ?? null,
    ]);

    return $stmt->fetch(PDO::FETCH_ASSOC);
}
```

Explication : findById() prend systématiquement \$userId en second paramètre, combiné à la clause WHERE : un utilisateur ne peut jamais accéder à la fiche d'un livre appartenant à quelqu'un d'autre, même en devinant un UUID valide — la vérification d'appartenance est intégrée à la requête elle-même.

Chapitre 10 — Structure des dossiers

10.1 Arborescence du projet

```
booknest/
├── Dockerfile
├── README.md
├── app/
│   ├── Config/          → Database.php, OpenApi.php
│   ├── Controllers/     → Auth, Book, Dashboard, Home, Library, Loan
│   ├── Core/            → App.php, Request.php, Response.php, Router.php
│   ├── Middleware/      → AuthenticateMiddleware, MiddlewareInterface
│   ├── Models/          → Book.php, User.php
│   ├── Repositories/    → Author, Book, Home, Loan, ReadingSession, User
│   ├── Services/        → CacheService, GutendexService, JwtService
│   └── Views/           → auth/, books/, home/, layouts/, library/,
                        loans/, pages/, users/
├── composer.json / composer.lock
├── database/migrations/ → 001 à 010, SQL versionné
├── docker/              → nginx.conf
├── docker-compose.yml
├── index.php · router.php
├── phpunit.xml
├── public/
│   ├── assets/{css,images,js}
│   ├── openapi.json
│   ├── swagger/         → Swagger UI statique
│   └── uploads/books/    → couvertures uploadées (ignoré par Git)
└── tests/               → suite PHPUnit
```

35 dossiers, 100 fichiers (hors vendor/ et .git/)

10.2 Rôle de chaque dossier

public/ est la seule racine exposée par nginx — protège tout le reste du code d'un accès direct via l'URL. app/Core contient le socle du framework maison (routage, requête/réponse). app/Repositories isole toute logique SQL, jamais mêlée aux contrôleurs. database/migrations trace l'évolution du schéma dans le temps de façon rejouable.

Chapitre 11 — Base de données PostgreSQL

11.1 Connexion et migrations

La connexion PDO est centralisée dans une classe Database dédiée. Les identifiants (hôte, port, base, utilisateur, mot de passe) proviennent de variables d'environnement lues via un .env, chargé au bootstrap de l'application.

11.2 Transactions

Les opérations impliquant plusieurs écritures liées — par exemple créer un emprunt tout en passant le statut du livre à « lent » — sont encapsulées dans une transaction PDO explicite (beginTransaction / commit / rollBack), garantissant qu'aucun état partiel ne subsiste en cas d'erreur.

```
$pdo->beginTransaction();

try {
    $insert = $pdo->prepare("
        INSERT INTO loans (user_id, book_id, borrower, lent_at)
        VALUES (:user_id, :book_id, :borrower, now())
    ");
    $insert->execute([
        'user_id' => $_SESSION['user']['id'],
        'book_id' => $bookId,
        'borrower' => $borrower,
    ]);

    // Le statut 'lent' existe dans l'ENUM reading_status
    $update = $pdo->prepare("
        UPDATE books SET status = 'lent', updated_at = now()
        WHERE id = :id AND user_id = :user_id
    ");
    $update->execute([
        'id' => $bookId,
        'user_id' => $_SESSION['user']['id'],
    ]);

    $pdo->commit();
} catch (\Throwable $e) {
    $pdo->rollBack();
}
```

Explication : Si l'une des deux écritures échoue, rollBack() annule l'ensemble : jamais un emprunt créé sans que le livre ne passe au statut « prêté », ni l'inverse.

Chapitre 12 — Cache Redis

12.1 Stratégie de cache

Redis sert exclusivement de cache : aucune donnée métier n'y réside sans exister d'abord en PostgreSQL. La stratégie est cache-aside — on tente d'abord la lecture du cache ; en cas d'absence, on interroge la base et on écrit le résultat avec un TTL.

12.2 Un client RESP réimplémenté

Plutôt que de dépendre de l'extension PECL redis comme boîte noire, le client a été réimplémenté directement au-dessus d'un socket TCP brut, en suivant le protocole RESP (Redis Serialization Protocol) — choix pédagogique pour comprendre le protocole sous-jacent.

```
public function remember(string $key, callable $callback, int $ttl = 300): mixed
{
    // cache-aside : on tente d'abord de lire le cache,
    // puis on génère et stocke la donnée si elle n'existe pas.
    $cached = $this->get($key);
    if ($cached !== null) {
        return $cached;
    }

    $value = $callback();
    $this->set($key, $value, $ttl);

    return $value;
}
```

Explication : `remember()` est la méthode principale utilisée par les contrôleurs : elle rend transparente la présence ou non du cache pour l'appelant. Si Redis est indisponible (connexion échouée au bootstrap), toutes les méthodes publiques échouent silencieusement (retour null/false), sans jamais lever d'exception ni interrompre la requête — un principe de résilience validé par les tests unitaires.

Chapitre 13 — Sécurité et Authentification

13.1 JWT + session hybride

BookNest combine un token JWT (stocké en cookie, signé HS256) et une session PHP classique, reconstruite à chaque requête à partir du token. Ce choix hybride bénéficie de la portabilité d'un JWT tout en gardant la simplicité d'accès de `$_SESSION` dans le reste du code.

```
public static function createToken(array $user, int $ttl = self::DEFAULT_TTL): string
{
    $secret = self::getSecret();
    $issuedAt = time();

    $payload = [
        'iss' => 'booknest',
        'sub' => (string) ($user['id'] ?? ''),
        'email' => $user['email'] ?? '',
        'display_name' => $user['display_name'] ?? '',
        'iat' => $issuedAt,
        'nbf' => $issuedAt,
        'exp' => $issuedAt + $ttl,
    ];

    return JWT::encode($payload, $secret, 'HS256');
}
```

Explication : Le TTL par défaut est de 86400 secondes (24h). Passé ce délai, tout token présenté est automatiquement rejeté par la bibliothèque firebase/php-jwt lors du décodage.

13.2 Middleware de protection des routes

```
public function handle(Request $request): void
{
    if (!in_array($request->uri, $this->protectedPaths, true)) {
        return;
    }

    if (!empty($_SESSION['user'])) {
        return;
    }

    header('Location: /login', true, 303);
    exit;
}
```

Explication : Le code HTTP 303 (plutôt que 301/302) force le navigateur à rejouer la requête suivante en GET, quelle que soit la méthode d'origine — pertinent car une route protégée comme `/books/delete` est en POST, et on ne veut pas qu'une redirection préserve accidentellement cette méthode vers `/login`.

13.3 Sécurité des données

- Mots de passe hachés via `password_hash()` (bcrypt par défaut), jamais stockés en clair.
- Requêtes préparées PDO systématiques — aucune concaténation de variable dans une requête SQL.
- Secrets (mots de passe base de données) externalisés via variables d'environnement, jamais codés en dur dans `docker-compose.yml`.

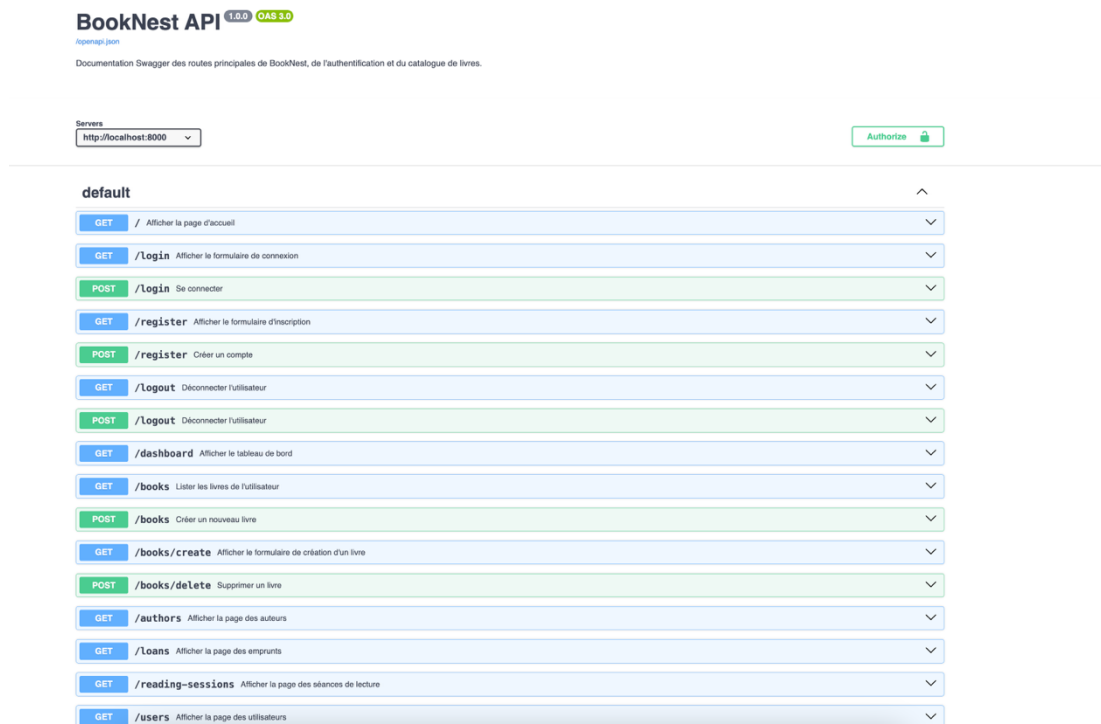
Chapitre 14 — API et Documentation Swagger

14.1 Génération depuis les annotations

Chaque route est documentée via des annotations PHPDoc directement dans les contrôleurs, scannées par zircote/swagger-php pour générer public/openapi.json, exposé via une interface Swagger UI statique à /swagger/.

```
/**
 * @OA\Post(
 *   path="/books/update-status",
 *   tags={"Books"},
 *   summary="Changer le statut de lecture d'un livre",
 *   @OA\RequestBody(
 *     required=true,
 *     @OA\JsonContent(
 *       required={"book_id","status"},
 *       @OA\Property(property="book_id", type="string", example="123e4567-e89b-12d3-a456-426614174000"),
 *       @OA\Property(property="status", type="string", enum={"to_read","reading","finished"})
 *     )
 *   ),
 *   @OA\Response(response=303, description="Statut mis à jour, redirection vers l'accueil")
 * )
 */
```

Explication : L'annotation vit au plus près du code qu'elle documente : toute modification de la route est immédiatement visible dans la documentation régénérée, sans fichier de spec séparé à maintenir manuellement.



Interface Swagger UI

14.2 Endpoints principaux

- POST /api/auth/register — inscription.
- POST /api/auth/login — obtention du JWT.
- GET /api/books — liste avec recherche et filtres.
- POST /api/books — ajout d'un livre (auth).
- GET /api/books/{id} — fiche détaillée.
- POST /api/loans — enregistrement d'un prêt.
- GET /api/stats — statistiques de lecture (cache 60 s).

Partie V — Front-end BookNest

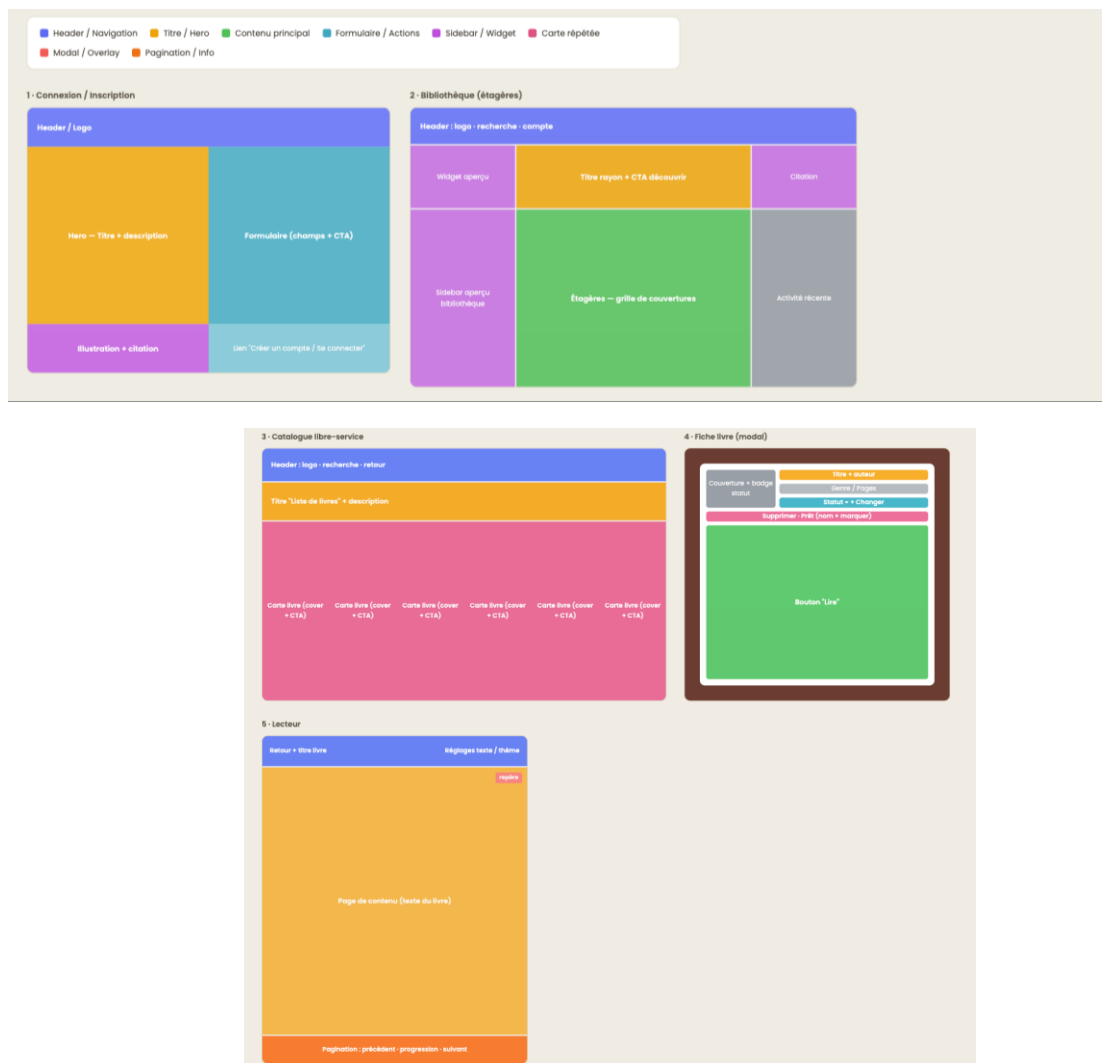
Chapitre 15 — Direction artistique

15.1 Une identité assumée

Plutôt qu'une interface générique de gestion de données, BookNest adopte une identité visuelle de bibliothèque ancienne : tons bois et laiton, étagères avec relief 3D (grain du bois, leur chaude ambiante), typographie à deux voix — Cormorant Garamond (serif) pour les titres et le texte du lecteur, Inter (sans-serif) pour l'interface.

15.2 Zoning

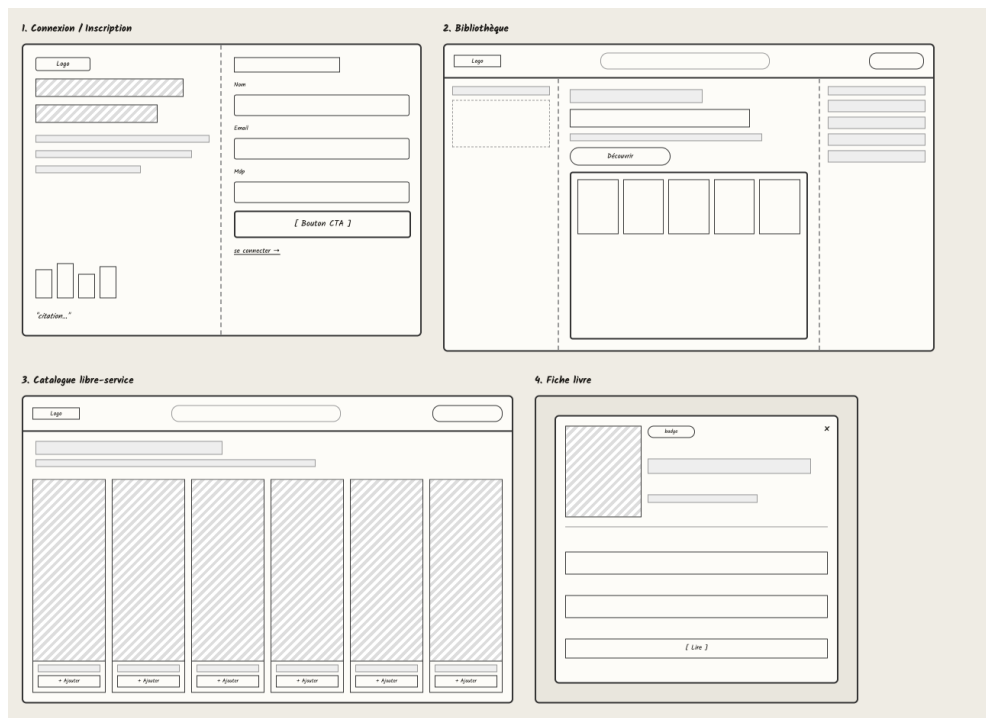
Découpage fonctionnel des écrans clés avant maquettage : chaque couleur représente un type de zone (header/navigation, titre/hero, contenu principal, formulaire/actions, sidebar/widget, carte répétée, modal/overlay, pagination/info), cohérent d'un écran à l'autre.



connexion/inscription et bibliothèque, catalogue libre service, fiche livre, lecteur

15.3 Wireframes

Structure basse-fidélité des 5 écrans principaux, sans couleur ni contenu final — pour valider l'agencement avant l'habillage visuel :



Wireframes : connexion, bibliothèque, catalogue, fiche livre

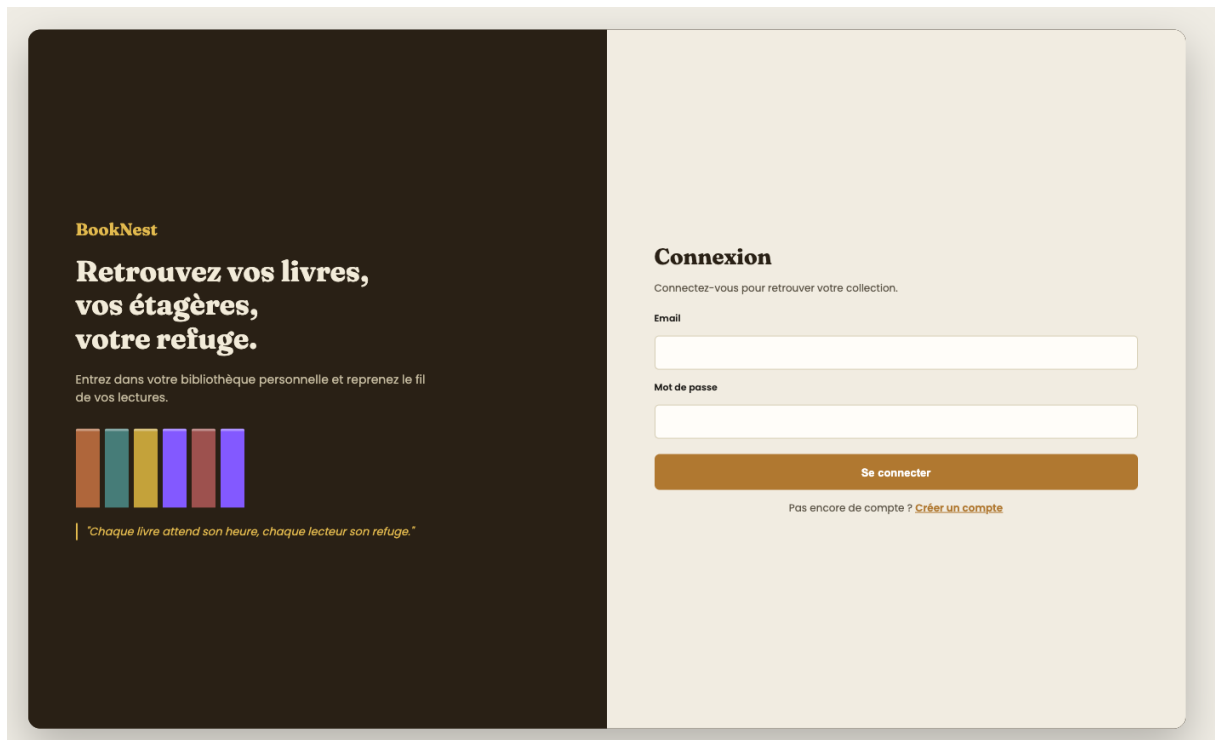


Wireframe du lecteur

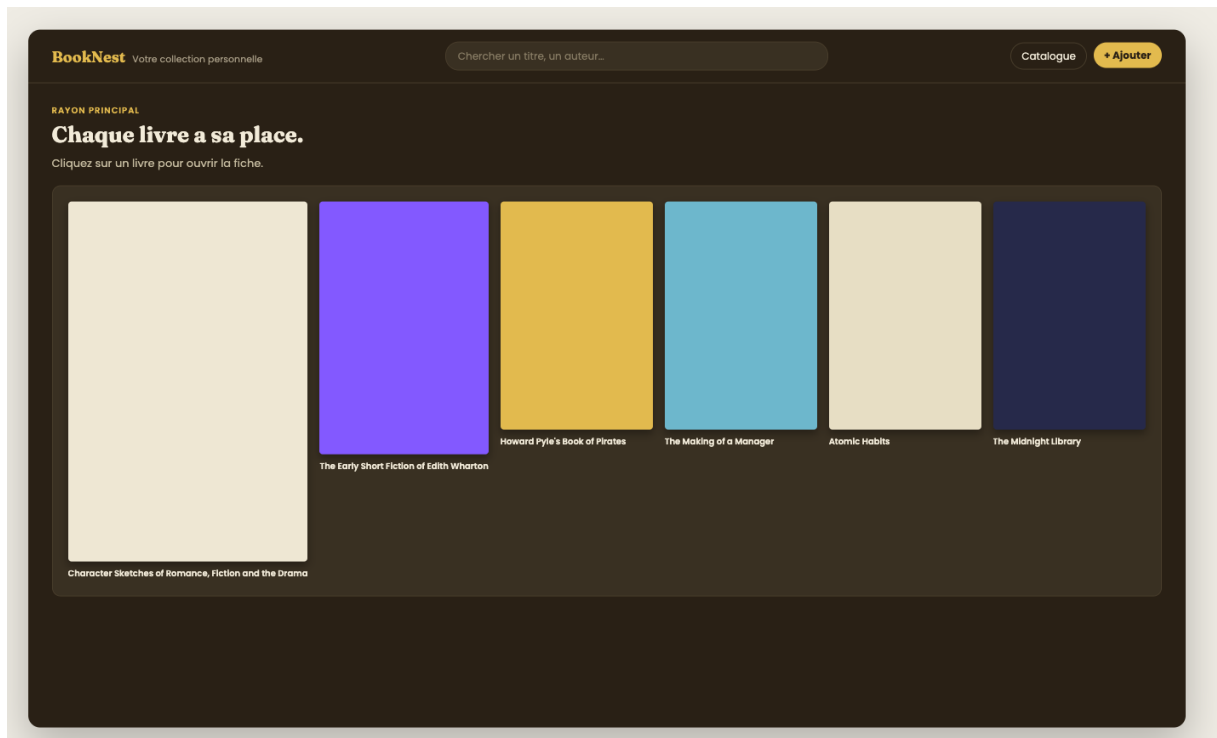
15.4 Palette de couleurs

Rôle	Couleur	Usage
Fond profond	#150b06 → #1f100a	Arrière-plan général
Crème parchemin	#f6ecd9 / #fbf5ea	Cartes, statistiques
Doré principal	#d7a563	Accents, bordures actives
Doré foncé	#b9803c	Hover, boutons primaires
Oxblood	#9c3b2e	Actions destructives

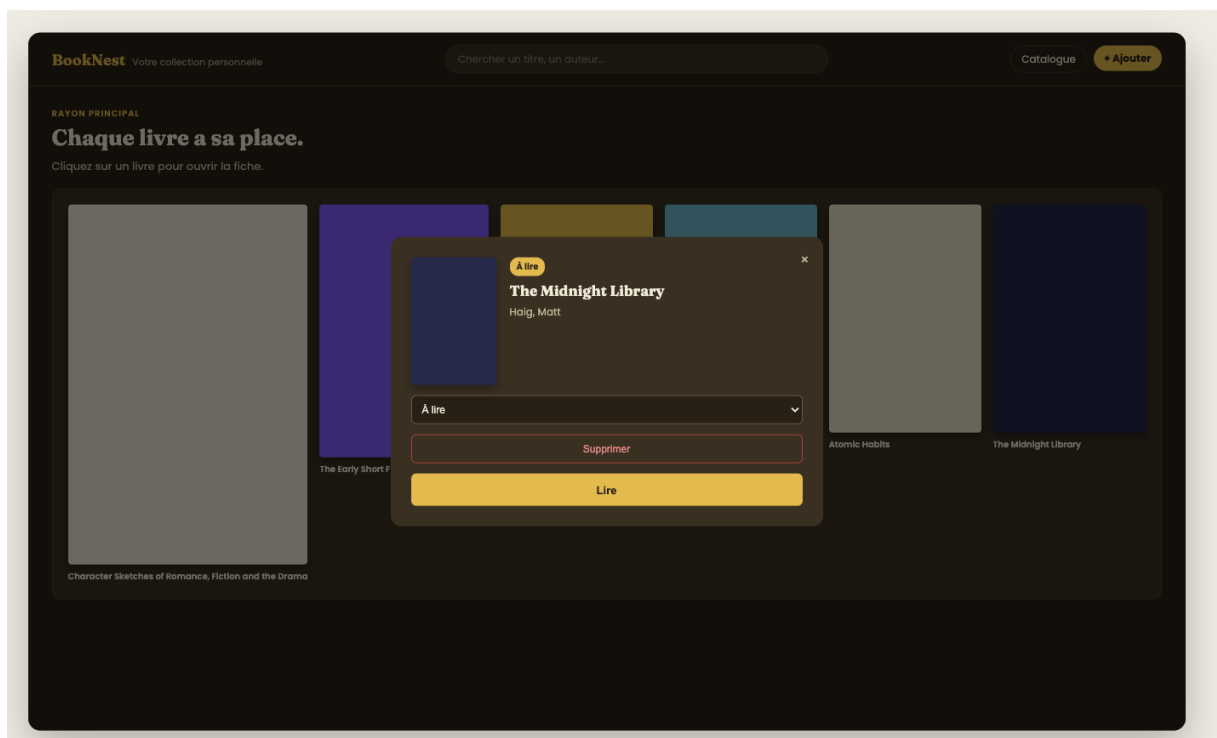
15.5 Prototype



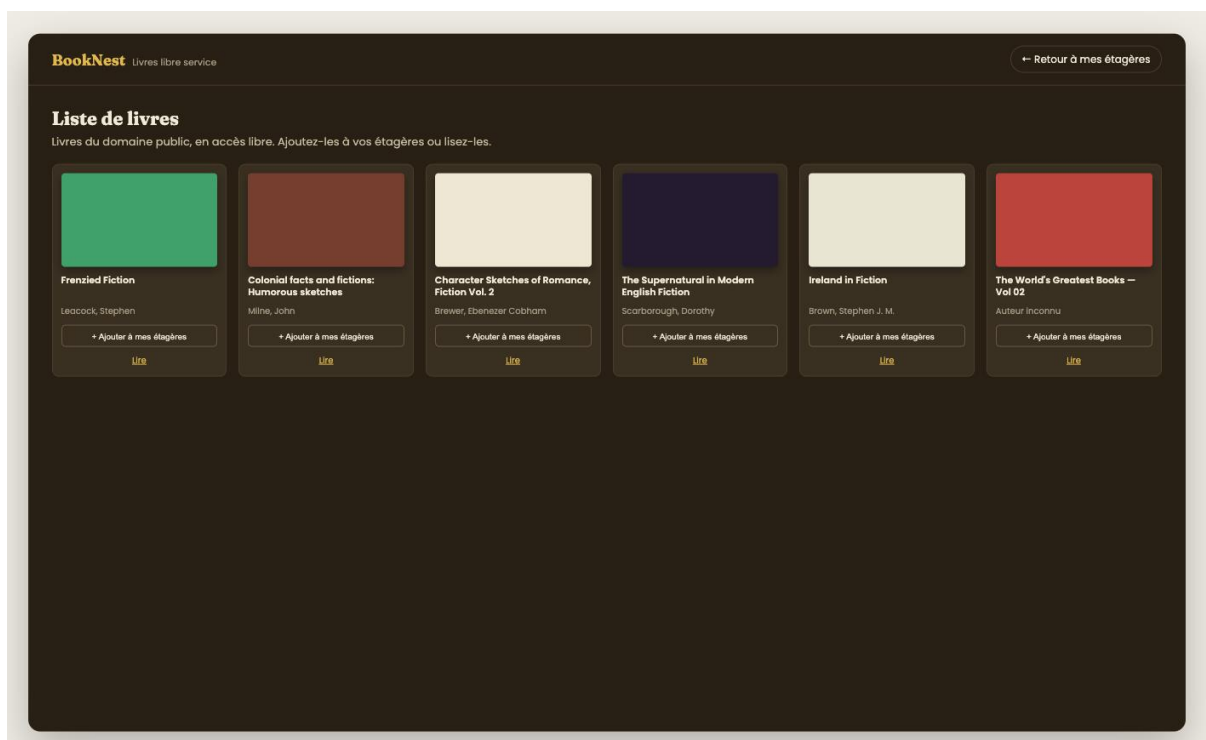
Page de connexion



Page d'accueil (étagères)



Fiche livre (modal)



Catalogue libre service

Chapitre 16 — Écrans principaux

16.1 Page d'accueil — étagères

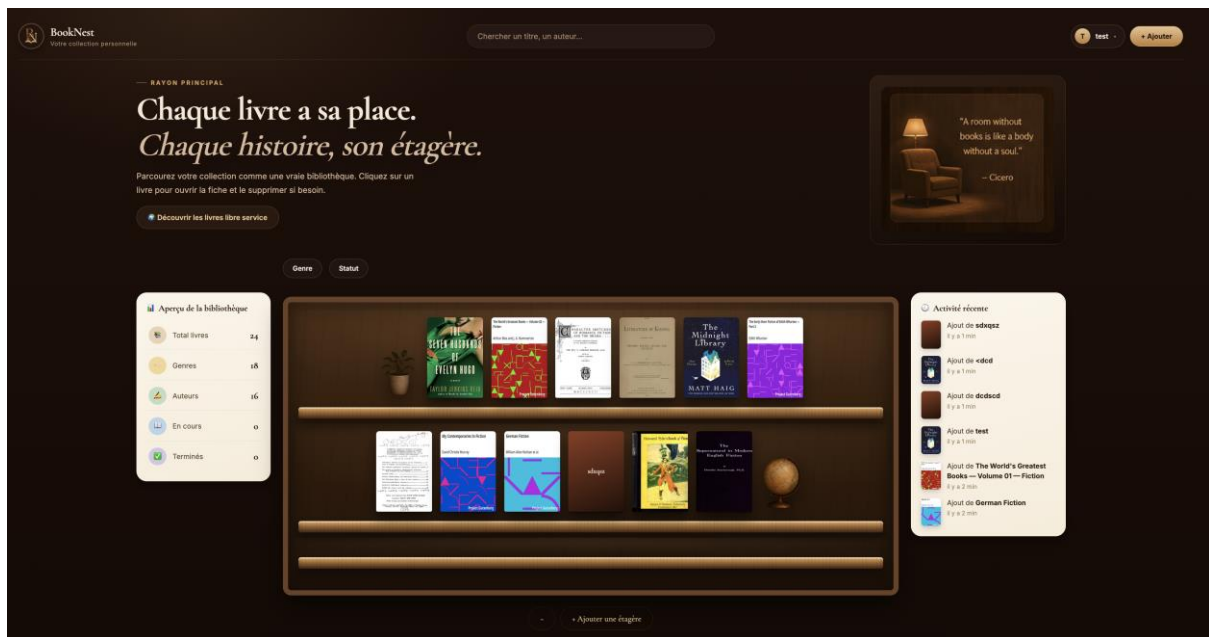


Figure 5 — Page d'accueil : bibliothèque en rayons

La page d'accueil dispose les livres sur plusieurs rayons.

16.2 Fiche livre

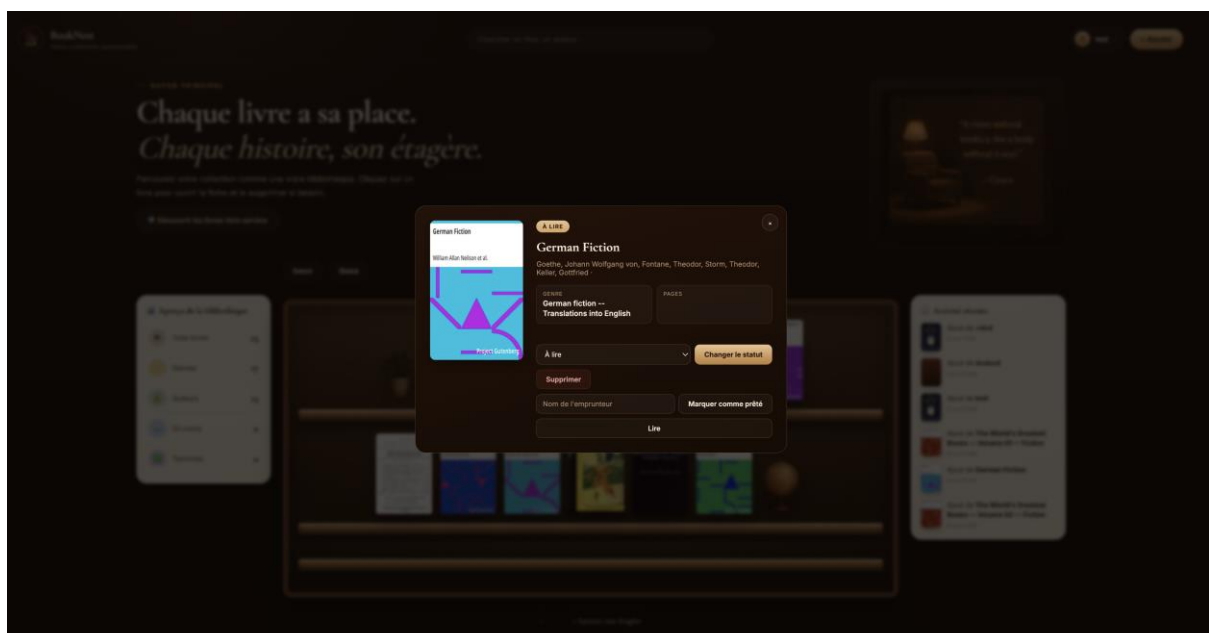


Figure 6 — Fiche livre au clic sur une tranche

Le dialogue affiche une couverture miniature, l'auteur, l'année, le statut, les notes personnelles, et propose des actions : lire le livre, changer de statut, supprimer, emprunt.

16.3 Lecteur immersif

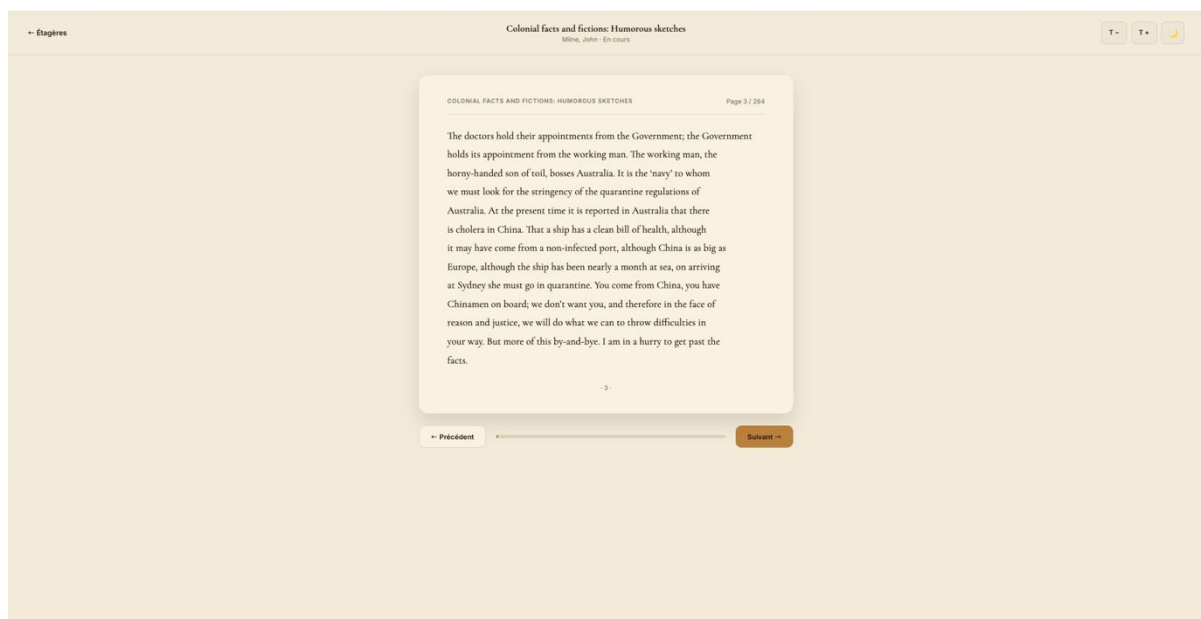


Figure 7 — Lecteur paginé avec thèmes

Le lecteur découpe le texte en pages, propose deux thèmes (sépia et nuit), un curseur de taille de police, et un marque-page qui reprend la couleur de la tranche du livre lu.

16.4 Formulaire d'ajout

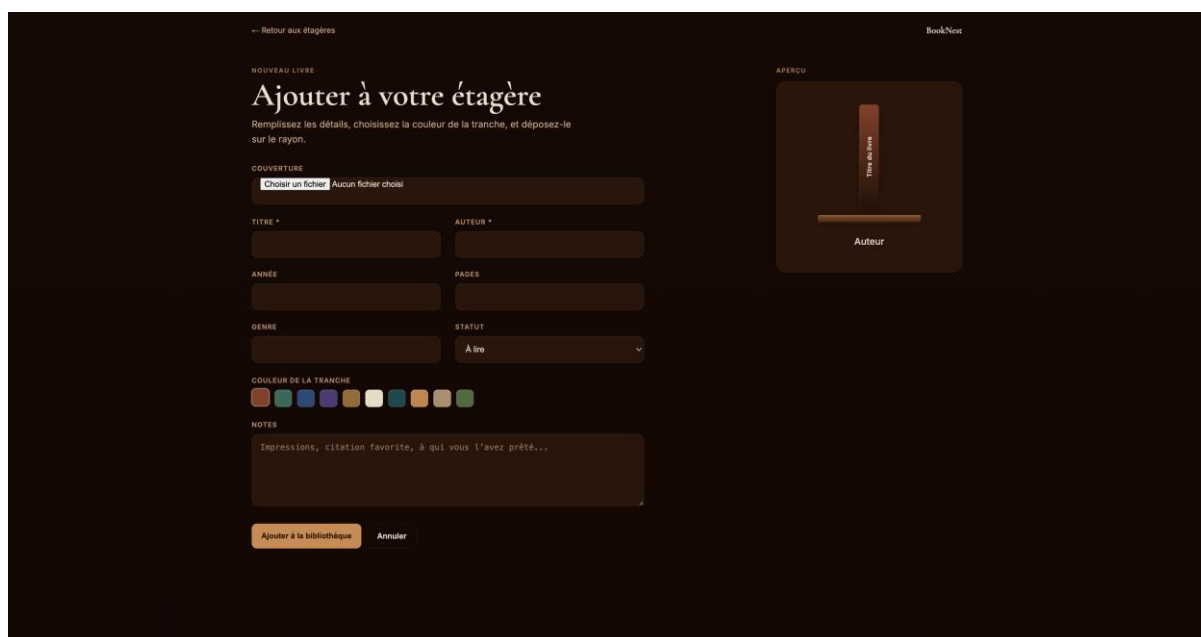
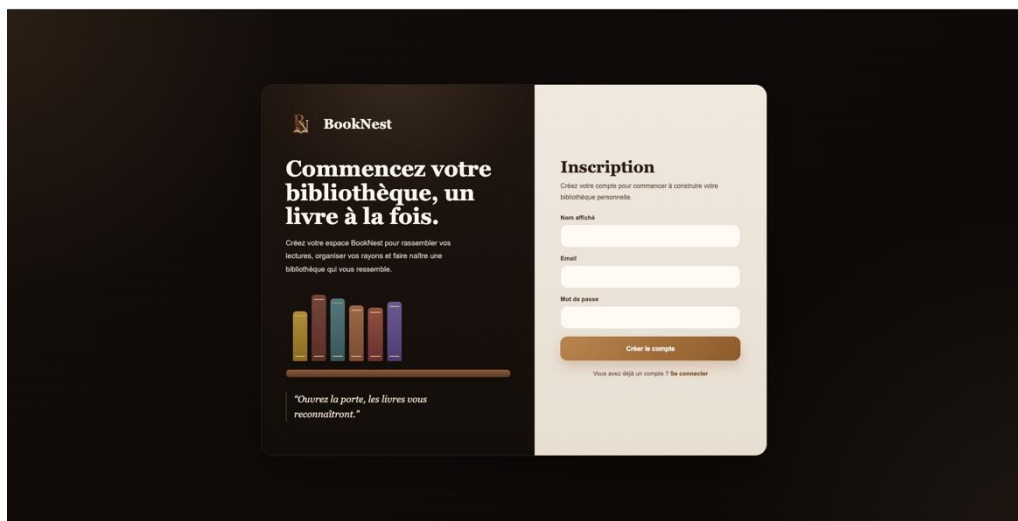
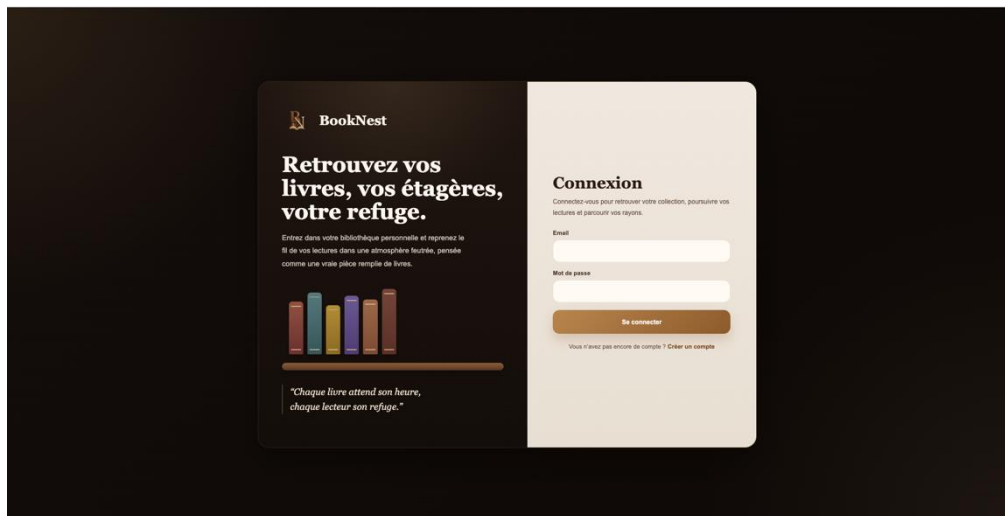


Figure 8 — Formulaire avec aperçu

16.5 Page de connexion/Inscription



Partie VI — Qualité & Livraison

Chapitre 17 — Tests

17.1 Tests automatisés

Suite PHPUnit couvrant les composants isolables du projet, avec une base SQLite en mémoire pour les repositories (pas de dépendance à PostgreSQL pour les tests) :

Fichier	Composant	Ce qui est vérifié
BookRepositoryTest.php	BookRepository	Création + lecture, suppression d'un livre et de sa couverture
CacheServiceTest.php	CacheService	Dégradation gracieuse si Redis indisponible
JwtServiceTest.php	JwtService	Cycle création/décodage, rejet d'un token invalide
RouterTest.php	Router	Dispatch correct, retour 404 pour route inconnue

```
Time: 00:00.215, Memory: 8.00 MB

Book Repository
✓ Create and all
✓ Delete removes row and file

Cache Service
✓ Cache unavailable returns null
✓ Remember uses callback when cache unavailable

Jwt Service
✓ Create and decode token
✓ Decode invalid token returns null

Router
✓ Dispatch calls correct route
✓ Dispatch returns 404 for unknown route

OK (8 tests, 19 assertions)
```

Résultats des tests unitaires

17.2 Cahier de recette

Authentification

Test	Objectif	Données d'entrée	Résultat attendu
Inscription	Créer un compte	Email, mot de passe, nom	Compte créé, redirection accueil
Connexion invalide	Rejeter	Mauvais mot de passe	Message d'erreur, pas de session
Déconnexion	Fermer la session	—	Session détruite, redirection /login

Livres & Emprunts

Test	Objectif	Données d'entrée	Résultat attendu
Ajout de livre	Créer un livre	Titre, auteur obligatoires	Livre visible sur l'étagère
Changement de statut	Modifier le statut	Nouveau statut valide	Livre déplacé sur la bonne étagère
Marquer comme prêté	Créer un emprunt	Nom de l'emprunteur	Emprunt créé, statut livre → lent

Marquer comme rendu	Clôturer un emprunt	ID du livre	returned_at renseigné, statut → to_read
---------------------	---------------------	-------------	--

Catalogue libre-service & étagères

Test	Objectif	Données d'entrée	Résultat attendu
Recherche Gutendex	Interroger l'API	Terme de recherche	Liste de résultats affichée
Ajout à l'étagère	Importer un livre externe	volume_id Gutendex	Livre créé avec google_volume_id
Lecture paginée	Charger le texte	volume_id + page	Page affichée, session de lecture démarrée
Ajouter une étagère	Incrémenter extra_shelves	—	Une étagère vide supplémentaire apparaît

Chapitre 18 — Gestion Git

18.1 Organisation des branches

Le projet est versionné sur GitHub. La branche par défaut est main ; le développement se fait sur develop et sur des branches de fonctionnalité dédiées, fusionnées progressivement :

main(branche par défaut, remote)

```
core/openapi-router
db/migrations
develop
feat/home-profile-logout
feat/home-search
feat/library-architecture
feature/bootstrap
feature/bootstrap-mvc
feature/library
middleware/authentication
```

feature/library(branche active à date de rédaction)

Explication : Chaque branche isole une préoccupation précise : mise en place du squelette MVC (feature/bootstrap*), authentification (middleware/authentication), évolution du schéma (db/migrations), routage et documentation Swagger (core/openapi-router), puis les fonctionnalités métier (feat/home-*, feature/library). Cette granularité permet de revenir sur une fonctionnalité isolément sans affecter le reste.

18.2 Convention de commits

```
feat(library): switch to Gutendex full-text reader instead of Google Books widget
feat(loans): add loan creation and return workflow
feat(home): redesign layout with overview/activity columns and shelf toolbar
fix(router): instantiate controller before route registration
chore(docker): increase nginx client_max_body_size for cover uploads
```

Explication : Cette granularité (feat/fix/chore) permet de retracer précisément l'évolution de chaque fonctionnalité et facilite un retour en arrière ciblé si nécessaire.

Chapitre 19 — Déploiement

19.1 Docker Compose

Explication : Les secrets (mot de passe PostgreSQL) sont externalisés via des variables d'environnement lues depuis `.env` (non versionné), plutôt que codés en dur — une correction apportée en cours de projet après revue du fichier.

```
services:
  >Run Service
  app:
    build: .
    volumes:
      - ./var/www/html:delegated
    depends_on:
      - db
    environment:
      - APP_ENV=development
    networks:
      - booknet

  >Run Service
  web:
    image: nginx:stable-alpine
    ports:
      - "8000:80"
    volumes:
      - ./var/www/html:ro
      - ./docker/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
    networks:
      - booknet

  >Run Service
  db:
    image: postgres:13
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - db-data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    networks:
      - booknet

  >Run Service
  redis:
    image: redis:7
    ports:
      - "6379:6379"
    networks:
      - booknet

volumes:
  db-data:

networks:
  booknet:
    driver: bridge
```

19.2 Migrations versionnées

Le schéma évolue via des scripts SQL numérotés (001 à 010), rejouables intégralement sur une base vierge :

```
for f in database/migrations/*.sql; do
  docker compose exec -T db psql -U postgres -d booknest < "$f";
done
```

19.3 Hébergement en ligne et intégration continue

Au-delà de l'environnement de développement local (Docker Compose sur la machine de développement), l'application est déployée sur Render, une plateforme d'hébergement gratuite pour les projets personnels. Le déploiement repose sur trois éléments :

- Un Dockerfile dédié (Dockerfile.render), qui combine nginx et PHP-FPM dans un seul conteneur via supervisord — Render exécute un conteneur unique par service, contrairement à Docker Compose en local qui orchestre plusieurs conteneurs séparés
- Un fichier render.yaml (Infrastructure as Code) qui décrit les trois services nécessaires — l'application web, une base PostgreSQL et un cache Redis — permettant de recréer tout l'environnement Render en un clic plutôt que de configurer chaque service manuellement
- Un pipeline d'intégration continue GitHub Actions qui exécute la suite de tests PHPUnit à chaque envoi sur la branche principale, et ne déclenche le déploiement sur Render que si l'ensemble des tests passe

```
name: CI/CD — Tests puis déploiement Render
on:
  push:
    branches: [main]

jobs:
  test:
    steps:
      - run: composer install
      - run: vendor/bin/phpunit --testdox

  deploy:
    needs: test
    if: success()
    steps:
      - run: curl -f "${{ secrets.RENDER_DEPLOY_HOOK_URL }}"
```

Explication : Le déploiement n'est jamais déclenché directement : il dépend systématiquement de la réussite préalable des tests automatisés (needs: test, if: success()). Une régression détectée par la suite PHPUnit bloque donc la mise en production avant même d'atteindre Render — une démarche DevOps où la qualité conditionne la livraison plutôt que d'être vérifiée après coup.

Limite assumée du plan gratuit Render : la base PostgreSQL expire automatiquement 30 jours après sa création (contrainte du tier gratuit), nécessitant de rejouer les migrations sur une base recréée périodiquement pour un usage prolongé au-delà de la démonstration.

Partie VII — Bilan

Chapitre 20 — Difficultés rencontrées et solutions

Cette section retrace, dans l'ordre chronologique, une chaîne de difficultés réellement rencontrées — depuis la remise en cause du choix d'API pour le catalogue libre service jusqu'aux bugs de performance et de cache découverts lors des derniers tests utilisateur. Elle illustre une démarche de résolution de problème complète : symptôme observé, diagnostic, correction, vérification.

20.1 Incompatibilité Docker Desktop / macOS

Problème : installation de Docker Desktop refusée (« Impossible d'utiliser cette version de l'application Docker avec cette version de macOS »), Docker Desktop ne supportant qu'un nombre limité de versions macOS récentes.

Solution : bascule vers Colima, moteur Docker léger basé sur une VM Lima, sans contrainte de version d'OS — installé simplement via Homebrew.

20.2 Bascule d'API externe pour le catalogue libre service

L'intégration initiale reposait sur l'API Google Books, avec lecture via son widget officiel. Problèmes rencontrés : quota anonyme rapidement épuisé, catalogue de résultats peu pertinent (vieux fonds de bibliothèque scannés plutôt que de véritables œuvres), et certains résultats passant le filtre « gratuit » sans être réellement lisibles intégralement (viewability différent de ALL_PAGES, widget affichant « Acheter ce livre »).

Solution : bascule complète vers l'API Gutendex (Project Gutenberg) — gratuite, sans clé ni quota, donnant accès au texte intégral brut de véritables œuvres du domaine public. Développement d'un lecteur de texte paginé fait maison plutôt que de dépendre d'un widget tiers : plus de contrôle sur le découpage en pages, plus de fiabilité.

20.3 Cache navigateur masquant les mises à jour CSS

Problème : après modification de la feuille de style, l'ancienne version restait affichée malgré un rafraîchissement forcé, à cause d'un en-tête Cache-Control trop agressif sur les fichiers statiques.

```
<link rel="stylesheet" href="/assets/css/home.css?v=<?= filemtime(
__DIR__ . '/../../public/assets/css/home.css'
) ?>">
```

Explication : Un paramètre de version basé sur la date de modification du fichier force le navigateur à recharger dès que le CSS change, sans intervention manuelle future.

20.4 Perte de la base de données après renommage du projet

Problème : renommage du dossier du projet (booknest → ProjetCDA), puis erreur PDOException relation "users" does not exist au premier chargement.

Diagnostic : le nom du dossier fait partie du nom des volumes Docker générés automatiquement par Docker Compose (booknest_db-data devient projetcda_db-data). Le renommage a donc silencieusement créé un tout nouveau volume vide, sans lien avec les données existantes — les migrations n'y avaient jamais été rejouées.

Solution : rejouer l'intégralité des migrations sur le nouveau volume :

```
for f in database/migrations/*.sql; do
  docker compose exec -T db psql -U postgres -d booknest < "$f";
done
```

20.5 Doublons de livres importés depuis le catalogue

Problème : un même livre du catalogue libre service se retrouvait parfois dupliqué dans la bibliothèque (deux lignes distinctes pour le même google_volume_id).

Diagnostic : LibraryController::add() insérait systématiquement une nouvelle ligne, sans vérifier si l'utilisateur avait déjà ce livre — un double-clic sur « Ajouter » suffisait à créer un doublon.

Solution : vérification de l'existence (user_id + google_volume_id) avant toute insertion, avec redirection silencieuse si le livre est déjà présent.

20.6 Lenteur perçue de la recherche et de la lecture

Problème : ouvrir le catalogue libre service ou un livre à lire prenait plusieurs secondes, à chaque fois, y compris pour un livre déjà consulté.

Diagnostic : LibraryController::index() et ::read() appelaient l'API Gutendex en direct à chaque requête, sans aucune mise en cache — y compris pour changer simplement de page dans un livre déjà chargé.

Solution : mise en cache des résultats de recherche (10 min) et des métadonnées de livre (24h) via CacheService, complétée par un stockage des métadonnées en session dès le premier accès — les changements de page (Précédent/Suivant) ne déclenchent alors plus aucun appel réseau ni requête de cache.

20.7 Cache Redis silencieusement inactif

Problème : après la correction précédente, la lenteur persistait exactement comme avant. Un test direct du cache (CacheService::set() puis ::get()) retournait systématiquement null.

Diagnostic : la variable d'environnement REDIS_HOST n'était pas définie dans .env. CacheService retombait alors sur sa valeur par défaut 127.0.0.1 — qui, depuis l'intérieur du conteneur app, pointe vers le conteneur lui-même et non vers le conteneur redis. Un problème de configuration identique, dans son principe, à celui déjà rencontré avec DB_HOST en tout début de projet (Chapitre 11).

Solution : ajout de REDIS_HOST=redis et REDIS_PORT=6379 dans .env — sur le réseau Docker interne, le nom du service tient lieu d'adresse.

20.8 Corruption du cache sur les grosses valeurs (texte de livre)

Problème : une fois Redis correctement configuré, la recherche du catalogue devenait rapide, mais la lecture d'un livre restait défailante.

Diagnostic : le client Redis du projet réimplémente le protocole RESP directement au-dessus d'un socket TCP (Chapitre 12). Or `fwrite()` et `fread()` sur un socket PHP ne garantissent pas de traiter la totalité d'un payload en un seul appel — une limite sans conséquence pour de petites valeurs (résultats de recherche), mais critique pour le texte intégral d'un livre, potentiellement plusieurs centaines de Ko.

```
// avant : un seul appel, silencieusement tronqué sur les gros payloads
$payload = fread($this->socket, $length + 2);

// après : boucle jusqu'à avoir lu exactement $length + 2 octets
$toRead = $length + 2;
$payload = '';
while (strlen($payload) < $toRead) {
    $chunk = fread($this->socket, $toRead - strlen($payload));
    if ($chunk === false || $chunk === '') { $this->available = false; return false; }
    $payload .= $chunk;
}
```

Explication : Le même principe est appliqué en écriture (`writeAll()`). Cette catégorie de bug — une limite d'API système qui ne se manifeste qu'au-delà d'une certaine taille de donnée — est particulièrement instructive : un test avec de petites valeurs ne l'aurait jamais révélée.

Chapitre 21 — Perspectives d'évolution

- Étendre la recherche sur les livres via l'extension `pg_trgm` (déjà installée, non encore exploitée par un index).
- Ajouter des tests d'intégration et end-to-end, au-delà des tests unitaires actuels.
- Mettre en place un pipeline CI/CD (GitHub Actions) exécutant la suite PHPUnit à chaque push.
- Étoffer le suivi des sessions de lecture (durée, pages lues) au-delà du simple horodatage début/fin.
- Reconstituer une charte graphique et des maquettes formalisées a posteriori, pour documenter la démarche UX/UI attendue.

Conclusion générale

BookNest démontre qu'une architecture MVC construite sans framework, associée à une conteneurisation Docker complète et à une intégration réfléchie d'une source de données externe, peut produire un outil cohérent et réellement utilisable. Le projet a permis de mettre en pratique l'ensemble de la chaîne de compétences visées par la certification : modélisation Merise, architecture logicielle en couches, sécurité applicative, tests automatisés, et démarche DevOps via Docker.

Bibliographie

- PHP: The Right Way — <https://phptherightway.com>
- Documentation PostgreSQL 13 — <https://www.postgresql.org/docs/13/>
- Documentation Redis / protocole RESP — <https://redis.io/docs/latest/develop/reference/protocol-spec/>
- Gutendex API — <https://gutendex.com>
- Spécification OpenAPI 3.0 — <https://spec.openapis.org/oas/v3.0.3>
- Merise — Nanci, Espinasse, Cohen, éd. Vuibert.

Conclusion générale

BookNest démontre qu'une application web professionnelle peut être à la fois techniquement rigoureuse — MVC strict, PostgreSQL, Redis, JWT, Swagger, tests PHPUnit — et visuellement chaleureuse. Le pari initial était de fuir l'esthétique CRUD au profit d'une métaphore immersive : le résultat, avec ses étagères boisées et son lecteur paginé, valide pleinement cette intuition.

Sur le plan pédagogique, ce projet a permis de mettre en pratique l'ensemble du cycle Merise, la conception d'une API REST documentée, la gestion Git professionnelle et le déploiement conteneurisé. Les perspectives d'évolution (import ISBN, mobile, recommandations) ouvrent de nombreuses pistes pour prolonger le travail.

BookNest est prêt à accueillir vos livres — et, qui sait, à devenir votre bibliothèque de tous les jours.